队伍编号	MCB2302632
赛道	（B）

基于大数据驱动的电商零售商品需求预测

摘 要

本文对**电商零售商品需求量问题**进行了深入研究，从问题深度来说，将问题分解为商品分类情况下的需求量预测，新增维度下的商品需求量预测，大型促销货物货量陡增情况下的商品需求量预测。

问题一：对于**商品分类情况下的需求量预测**，本质上是一个时间序列预测问题。针对不同商家不同商品不同仓库的 **1996 种组合**，利用原始的 **166 天**销售量数据，基于**销售量的统计特征**（最小值，最大值，中位数，上四分位数，下四分位数，平均值，标准差）进行 **k-均值聚类**，从**轮廓系数和轮廓图**综合考虑将 1996 种组合聚成了 **4 类**，从**模型的简易性，实用性，与可解释性**方面放弃了回归预测，灰色预测与深度学习模型，针对每一类选取合适的 **ARIMA 预测模型**进行为期 15 天的销售量预测。以组合（seller_2,product_65,warehouse_1）为例进行模型求解展示，计算得到 **1-wmape 为 0.5448**。

问题二：对于新增维度下的商品需求量预测，本质同上题，为一个时间序列预测问题。相比于问题一特点是组合数量只有 **210 种**，并且时间维度也不一样，历史数据有 36 天，28 天，45 天等等。本文利用原始的不同维度销售量数据，并基于问题 1 的模型，计算 **210 种组合和问题 1 的 4 个类别中心之间的距离**，将**组合归为距离最近的类别中心对应的类别**，然后针对每一类选取合适的 **ARIMA 预测模型**进行为期 15 天的销售量预测。

问题三：对于大型促销货物货量陡增情况下的商品需求量预测，本质同前两问，仍是一个时间序列预测问题。该问题组合数量有 **1957 种**，时间维度大多数是 11 天，考虑到在问题一中，已经预测出 2023-05-16 到 2023-05-30 的销售量，但由于预测值本身存在误差，再作为原始数据来预测 6 月的数据就会产生误差叠加的后果。因此本文利用双 11 期间的 **11 天**原始销售量数据，并基于问题 1 的方法**先进行聚类**，再针**对每一类选取合适的 ARIMA** 预测模型进行为期 9 天的销售量预测，然后基于**预测模型的平均残差**对 2022 年的 20 天数据进行**调整**，得到 2023-06-01 到 2023-06-20 的预测值。

针对预测模型，本文做了严格的**模型检验**，包括**平稳性检验，白噪声检验与残差检验**。相比于深度学习模型，建立的时间序列预测模型具有**简易性与适用性**。

关键词：k-均值聚类 轮廓图 ARIMA 时间序列分析 平稳性检验 白噪声检验与残差检验

目录

1．问题背景与问题重述 1
1.1 问题背景 1
1.2 问题重述 1
2．模型假设与符号说明 2
2.1 模型假设 2
2.2 符号说明 2
3．数据质量检测 3
4．问题一的分析及模型建立与求解 5
4.1 问题分析与分解 5
4.2 模型建立与求解 5
4.2.1 k-均值模型建立 5
4.2.2 k-均值模型求解 7
4.2.3 时间序列预测模型建立 10
4.2.4 时间序列预测模型求解 12
4.3 模型小结和分析 14
5．问题二的分析及模型建立与求解 15
5.1 问题分析与分解 15
5.2 模型建立 16
5.3 模型求解 18
5.4 模型小结和分析 19
6．问题三的分析及模型建立与求解 19
6.1 问题分析与分解 19
6.2 模型建立 20
6.3 模型求解 21
6.4 模型小结和分析 22
7．模型的评价与改进 22
7.1 模型优点 22
7.2 模型缺点与改进 23
参考文献 23
附录 24

1．问题背景与问题重述

1.1 问题背景

电商平台存在着上千个商家，他们会将商品货物放在电商配套的仓库，电商平台会对这些货物进行统一管理。通过科学的管理手段和智能决策，**大数据智能驱动**的供应链可以显著降低库存成本，同时保证商品的按时履约。

电商平台需要提前预知各地的需求，从而将库存提前放在靠近需求的仓库中，此时的预测任务为：根据历史一段时间的需求量，预测各仓库中各商品未来需求，**“预测维度”即为不同商家在各仓库中存放的各种商品每天的数量**。企业会首先根据数据的历史情况，分析出需求量序列的数理特征，对相似的需求量序列进行归类，并根据分类结果做到更加精准的预测。然而，在实际的电商供应链预测任务中，常常会出现多种不同的问题。例如，部分商品的销售时间过短、仓库存在新增或切换等情况，导致该预测维度下历史数据过少；另外，部分大型促销期间货量的陡增并由此带来的不规律性，也给需求量的精准预测带来了不小的难度。此时便需要通过算法得到历史一般规律，找出相似的历史情况（如相似的仓或商品），从而实现精准预测。

1.2 问题重述

基于上述问题背景，建立数学模型求解如下三个问题：

- **问题一：商品分类情况下的需求量预测**

使用附件 1-4 中的数据，预测出各商家在各仓库的商品 2023-05-16 至 2023-05-30 的需求量，将预测结果填写在结果表 1，并对模型的预测性能进行评价。此外，本问还需要依据数据分析及建模过程，实现对商家、仓库、商品形成的时间序列行分类，使同一类别在需求上的特征最为相似。

- **问题二：新增维度下的商品需求量预测**

现有一些新出现的商家+仓库+商品维度（附件 5），导致这种情况出现的原因可能是新上市的商品，或是改变了某些商品所存放的仓库。本问需要分析这些新出现的预测维度如何实现对历史附件 1 中的数据的参考，找到相似序列并完成这些维度在 2023-05-16 至 2023-05-30 的预测值。并把预测结果填写在结果表 2。

- **问题三：大型促销货物货量陡增情况下的商品需求量预测**

每年 6 月会出现规律性的大型促销，为需求量的精准预测以及履约带来了很大的挑战。附件 6 给出了附件 1 对应的商家+仓库+商品维度在去年双十一期间的需求量数据，本问需参考这些数据，给出 2023-06-01 2023-06-20 的预测值。并把预测结果填写在结果表 3。

提供的数据见表 1-1：

表 1-1 数据目录

数据名称	数据内容
附件 1-商家历史出货量表.xlsx	不同商家在不同仓库的不同商品的历史销售量
附件 2-商品信息表.xlsx	不同商品编码及一级，二级，三级分类类别
附件 3-商家信息表.xlsx	不同商家编码及分类类别
附件 4-仓库信息表.xlsx	不同仓库编码及分类类别，仓库区域
附件 5-新品历史出货量表.xlsx	新增商品的历史销售量
附件 6-促销期间商家出货量表.xlsx	促销期间商品的历史销售量

2．模型假设与符号说明

2.1 模型假设

为了构建相关数学模型，给出如下模型假设：

（1）**数据真实性假设**：通过假设所提供的数据是准确和可靠的，没有严重的错误或缺失；

（2）**数据分组假设**：假设将不同商家、仓库、商品形成的时间序列分类后，同类的销售量规律大致一样；

（3）**预测模型假设**：假设对同一组的销售量进行预测时，可用同一种预测模型。

2.2 符号说明

为了更清晰地阐述本论文所构建的数学模型，在此给出所用到的模型变量及其相关含义。

表 2-1 符号说明

模型变量	变量含义	备注说明
d_{ij}	样本i, j之间的距离	/
S	轮廓系数	取值[-1,1]
x_t	时间序列在t时刻的取值	/
φ	AR 模型的系数	/
θ	MA 模型的系数	/
a	白噪声序列	/
p	AR 模型的阶数	/
d	差分次数	/
q	MA 模型的阶数	/
B	延迟算子	/
∇	差分运算	/
λ	常数项	/
ε_t	误差项	/

3．数据质量检测

➢ 读取数据集并将数据按顺序排列

分析附件 1 到附件 6 数据，可以发现特征“seller_no”，“product_no”，“warehouse_no”，“date”都是有顺序的，因此通过将数据集按顺序排列。以“附件 1：商家历史出货量表”数据集为例子，首先通过借助正则表达式，将特征“seller_no”，“product_no”“warehouse_no”里面的数字提取出来，然后将数据集按照特征(优先级从高低)“seller_no”，“product_no”，“warehouse_no”，“date”排升序，附件 2 到附件 6 数据同理，前 5 行数据如表 3-1 所示，处理流程图如图 3-1 所示。

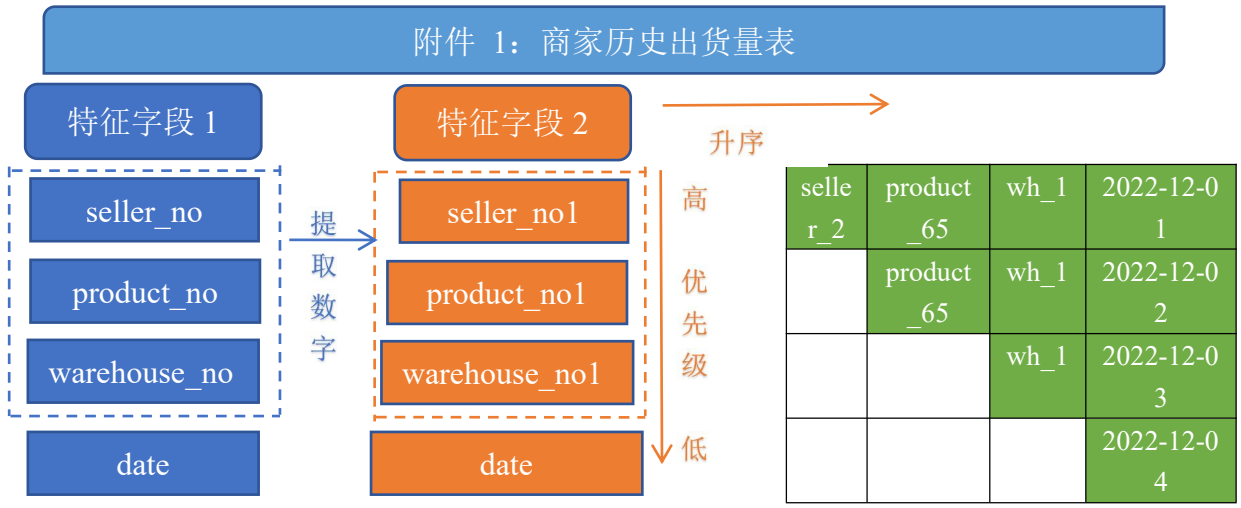


图 3-1 数据集升序排列处理流程

表 3-1 排序后数据集前 5 行

seller_no	product_no	warehouse_no	date	qty	seller_no1	product_no1	warehouse_no1
seller_2	product_65	wh_1	2022/12/1	3	2	65	1
seller_2	product_65	wh_1	2022/12/2	2	2	65	1
seller_2	product_65	wh_1	2022/12/3	3	2	65	1
seller_2	product_65	wh_1	2022/12/4	2	2	65	1
seller_2	product_65	wh_1	2022/12/5	7	2	65	1

➢ 删除重复值

利用 python 进行分析，发现只有“附件 4：仓库信息表”含有重复值，如表 3-2 所示，只保留第一个重复值。

表 3-2 附件 4 重复值情况

warehouse_no	warehouse _category	warehouse _region
wh_29	区域仓	华东
wh_29	区域仓	华东
wh_38	区域仓	西南
wh_38	区域仓	西南

➢ 填补缺失值

利用 python 进行分析，发现没有缺失值。

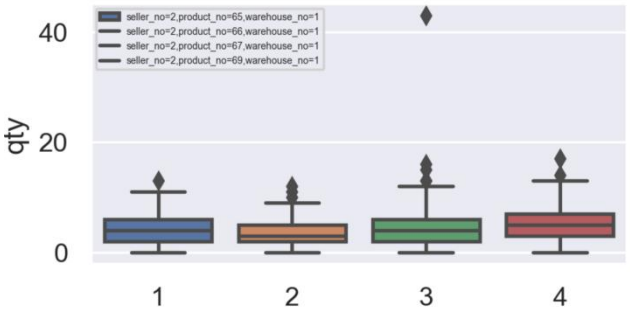
- 修改异常值

首先通过分析附件 3 商家编码，可以发现一共有 37 个商家，编号是从 seller_1 到 seller_37，这与附件 3 的数据集尺寸（37，4）是一致的；然后分析了附件 2 商品编码，一共有 2302 件商品，编号是从 product_1 到 product_2438（一些中间编号如 product_11 是没有的），然后分析了附件 4 仓库编码，一共是有 54 个仓库，编号是从 wh_1 到 wf_54。

然后再对附件 1，附件 5，附件 6 数据进行分组聚合，通过以前 4 组数据为例，绘制出箱线图来判断异常值情况。由于附件 5 维度不一，因此后面单独分析。

			qty							
			count	mean	std	min	25%	50%	75%	max
seller_no1	product_no1	warehouse_no1								
2	65	1	166.0	4.277108	2.725028	0.0	2.00	4.0	6.0	13.0
	66	1	166.0	3.433735	2.387811	0.0	1.00	3.0	5.0	12.0
	67	1	166.0	4.734940	4.596664	0.0	2.00	4.0	6.0	43.0
	69	1	166.0	5.198795	2.824610	0.0	3.00	5.0	7.0	17.0
		4	166.0	1.048193	1.184789	0.0	0.00	1.0	2.0	6.0
...	...	...	...	...	...	...	...	...	...	...
37	1461	8	166.0	1.698795	1.840520	0.0	0.00	1.0	3.0	9.0
	1462	8	166.0	4.012048	3.423604	0.0	1.25	3.0	6.0	16.0
	1463	37	166.0	1.198795	1.659572	0.0	0.00	1.0	2.0	9.0
	1464	10	166.0	1.481928	2.108347	0.0	0.00	1.0	2.0	11.0
	1465	10	166.0	11.512048	7.136537	0.0	7.25	11.0	15.0	43.0

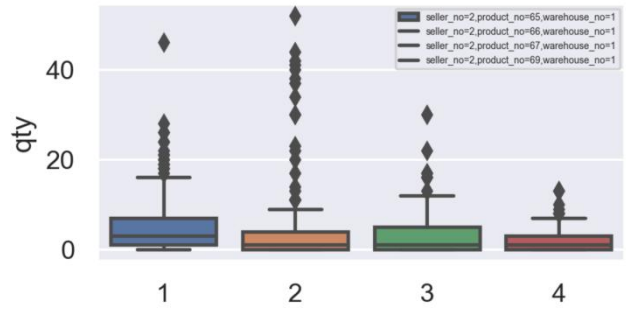
图（a）附件 1 数据分组聚合表



图（b）附件 1 示例数据箱线图

			qty							
			count	mean	std	min	25%	50%	75%	max
seller_no1	product_no1	warehouse_no1								
2	65	1	11.0	2.818182	1.834022	0.0	1.5	2.0	4.5	5.0
	66	1	11.0	3.363636	2.693426	0.0	1.5	3.0	4.5	8.0
	67	1	11.0	3.818182	2.442056	0.0	2.5	4.0	5.0	8.0
	69	1	11.0	3.363636	2.062655	2.0	2.0	3.0	3.0	9.0
		4	11.0	0.545455	0.522233	0.0	0.0	1.0	1.0	1.0
...	...	...	...	...	...	...	...	...	...	...
37	1461	8	11.0	2.363636	3.264130	0.0	0.0	1.0	2.5	10.0
	1462	8	11.0	3.000000	3.847077	0.0	0.0	2.0	4.0	11.0
	1463	37	11.0	0.090909	0.301511	0.0	0.0	0.0	0.0	1.0
	1464	10	11.0	0.818182	1.250454	0.0	0.0	0.0	1.5	3.0
	1465	10	11.0	19.454545	8.358991	8.0	14.5	19.0	22.0	36.0

图（c）附件 6 数据分组聚合表



图（d）附件 6 示例数据箱线图

图 3-2 数据分组聚合表与异常值分析箱线图

对于异常值，如果有特别的含义（比如双十一期间需求量骤增，异常值代表了特殊的节日订单量）那么在后续模型建立过程中需要特别考虑，如果没有特别的含义，那么将小的异常值用下四分位数替代，将大的异常值用上四分位数代替。

4．问题一的分析及模型建立与求解

4.1 问题分析与分解

问题一要求基于附件 1-4 中的数据，预测出各商家在各仓库的商品 2023-05-16 至 2023-05-30 的需求量并对模型的预测性能进行评价。最重要的是，需要考虑商家、仓库、商品形成的时间序列如何分类，使同一类别在需求上的特征最为相似。

问题一本质是一个**时间序列预测问题**，通过前面简单的数据质量检测发现，不同商家不同商品不同仓库的组合有 1996 种，也就是要预测 1996 次。对于商家、仓库、商品形成的时间序列分类问题，可以选取销售量的统计特征（如最小值，最大值，中位数，上四分位数，下四分位数，平均值，标准差）进行聚类，然后针对每一类选取合适的预测模型进行为期 15 天的销售量预测。

本问题的核心和难点是：

针对 1996 种组合（不同商家不同商品不同仓库），如何进行分类并对每一类选择合适的时间序列模型进行预测，来使得模型的精度更高。

问题 1 的求解分为如下几个关键步骤，求解流程图如下：

（1）首先数据提取，对附件 1 中的 1996 种组合（不同商家不同商品不同仓库）进行销售量数据提取。

（2）然后选取特征，计算 1996 种组合的统计特征如最小值，最大值，中位数，上四分位数，下四分位数，平均值，标准差。

（3）然后进行聚类，将上述的统计特征作为聚类特征，选择 k−均值模型进行聚类分析。

（4）然后进行预测，根据预测指标 1-wmape 为每一类选取合适的预测模型，使得预测指标尽可能接近于 1。

（5）最后对模型进行小结和分析。

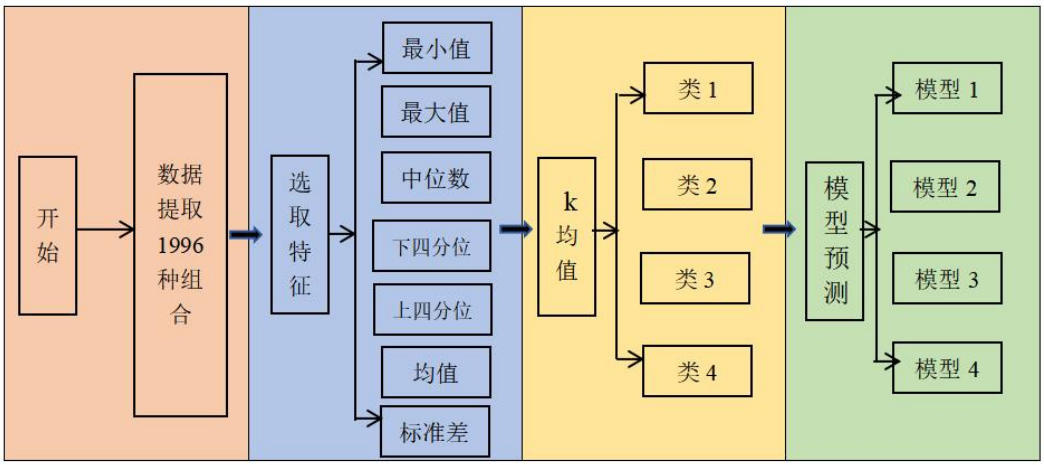


图 4-1 问题 1 整体思路图

4.2 模型建立与求解

4.2.1 k−均值模型建立

K-均值模型采用“物以类聚，人以群分”的思想，将距离近的归为一类[1]。

➢ 聚类准则

对连续性特征，通过以下距离来度量样本间的相似性：

表 4-1 距离公式

距离	定义式	说明
绝对值距离	$d_{ij}(1)=\sum_{k=1}^{p}\lvert x_{ik}-x_{jk}\rvert$	一维空间下进行的距离计算
欧式距离	$d_{ij}(2)=\sqrt{\sum_{k=1}^{p}(x_{ik}-x_{jk})^2}$	二维空间下进行的距离计算
闵可夫斯基距离	$d_{ij}(q)=[\sum_{k=1}^{p}(x_{ik}-x_{jk})^q]^{1/q},q>0$	q 维空间下进行的距离计算
切比雪夫距离	$d_{ij}(\infty)=\max_{1\le k\le p}\lvert x_{ik}-x_{jk}\rvert$	q 取无穷大时的闵可夫斯基距离
Lance 距离	$d_{ij}(L)=\sum_{k=1}^{p}\frac{\lvert x_{ik}-x_{jk}\rvert}{x_{ik}+x_{jk}}$	削弱极端值的影响能力

对离散型特征，采用 VDM 距离来度量样本之间的相似性：

令 $m_{u,a}$ 表示在属性 u 上取值为 a 的样本数，$m_{u,a,i}$ 表示在第 i 个样本簇中在属性 u 上取值为 a 的样本数，k 为样本簇数，则属性 u 上两个离散值 a 和 b 之间的 VDM 距离为：

$$VDM_p(a,b)=\sum_{i=1}^{k}\left|\frac{m_{u,a,i}}{m_{u,a}}-\frac{m_{u,b,i}}{m_{u,b}}\right|^p \tag{4.1}$$

对混合性特征，采用上述距离结合的方式来度量样本之间的相似性：

将闵可夫斯基距离和 VDM 结合即可处理混合属性，假定有 n_c 个有序属性，$n-n_c$ 个无序属性，不失一般性，令有序属性排列在无序属性之前，则

$$MinkovDM_p(x_i,x_j)=(\sum_{u=1}^{n_c}\lvert x_{iu}-x_{ju}\rvert^p+\sum_{u=n_c+1}^{n}VDM_p(x_{iu},x_{ju}))^{1/p} \tag{4.2}$$

➢ 聚类指标

由于没有真实标签，因此这里采用轮廓系数作为算法性能评价指标，包含两种因素，内聚度和分离度。内聚度可以理解为反映一个样本点与类内元素的紧密程度。分离度可以理解为反映一个样本点与类外元素的紧密程度[2]。

轮廓系数的公式为：

$$S(i)=\frac{b(i)-a(i)}{\max\{a(i),b(i)\}} \tag{4.3}$$

其中，$a(i)$ 表示样本点的内聚度，计算公式为

$$a(i)=\frac{1}{n}\sum_{j\neq i}^{n}d_{ij} \tag{4.4}$$

$b(i)$ 的计算方式与 $a(i)$ 类似。只不过需要遍历其他类簇得到多个值，从中选择最小的值作为最终的结果。

- 算法伪代码

给定样本集 $D=\{x_1,x_2,...,x_m\}$，K 均值针对聚类所得簇划分为 $C=\{C_1,...,C_k\}$，最小化平方误差：$E=\sum_{i=1}^{k}\sum_{x\in C_i}\|x-\mu_i\|_2^2$，其中 $\mu_i=\frac{1}{|C_i|}\sum_{x\in C_i}x$ 是簇 C_i 的均值向量，直观来看，误差刻画了簇内样本围绕簇均值向量的紧密程度，越小则簇内相似性越高。

输入： 样本集 $D=\{\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_m\}$;
聚类簇数 k.

过程：

1: 从 D 中随机选择 k 个样本作为初始均值向量 $\{\boldsymbol{\mu}_1,\boldsymbol{\mu}_2,\ldots,\boldsymbol{\mu}_k\}$
2: **repeat**
3: 令 $C_i=\varnothing\ (1\leqslant i\leqslant k)$
4: **for** $j=1,2,\ldots,m$ **do**
5: 计算样本 $\boldsymbol{x}_j$ 与各均值向量 $\boldsymbol{\mu}_i\ (1\leqslant i\leqslant k)$ 的距离：$d_{ji}=\|\boldsymbol{x}_j-\boldsymbol{\mu}_i\|_2$;
6: 根据距离最近的均值向量确定 $\boldsymbol{x}_j$ 的簇标记：$\lambda_j=\arg\min_{i\in\{1,2,\ldots,k\}}d_{ji}$;
7: 将样本 $\boldsymbol{x}_j$ 划入相应的簇：$C_{\lambda_j}=C_{\lambda_j}\bigcup\{\boldsymbol{x}_j\}$;
8: **end for**
9: **for** $i=1,2,\ldots,k$ **do**
10: 计算新均值向量：$\boldsymbol{\mu}_i'=\frac{1}{|C_i|}\sum_{\boldsymbol{x}\in C_i}\boldsymbol{x}$;
11: **if** $\boldsymbol{\mu}_i'\neq\boldsymbol{\mu}_i$ **then**
12: 将当前均值向量 $\boldsymbol{\mu}_i$ 更新为 $\boldsymbol{\mu}_i'$
13: **else**
14: 保持当前均值向量不变
15: **end if**
16: **end for**
17: **until** 当前均值向量均未更新

输出： 簇划分 $C=\{C_1,C_2,\ldots,C_k\}$

图 4-2 K-均值伪代码

4.2.2 k-均值模型求解

选取的特征为最小值，最大值，中位数，上四分位数，下四分位数，平均值，标准差，前 10 个示例数据如表所示：

表 4-2 聚类特征选择

mean	std	min	25%	50%	75%	max
4.277108	2.725028	0.0	2.00	4.0	6.0	13.0
3.433735	2.387811	0.0	1.00	3.0	5.0	12.0
4.734940	4.596664	0.0	2.00	4.0	6.0	43.0
5.198795	2.824610	0.0	3.00	5.0	7.0	17.0
1.048193	1.184789	0.0	0.00	1.0	2.0	6.0
1.337349	1.616409	0.0	0.00	1.0	2.0	14.0
4.301205	4.871027	0.0	1.00	3.0	6.0	43.0
19.698795	14.649220	0.0	12.00	17.0	23.0	97.0
17.819277	16.729321	0.0	7.25	14.5	21.0	111.0
1.765060	3.592785	0.0	0.00	0.0	2.0	26.0

将 1996 种组合分别聚成 4，5，6，7 类的轮廓系数为：**0.8486**，0.7810，0.7430，0.7472，绘制出轮廓图：

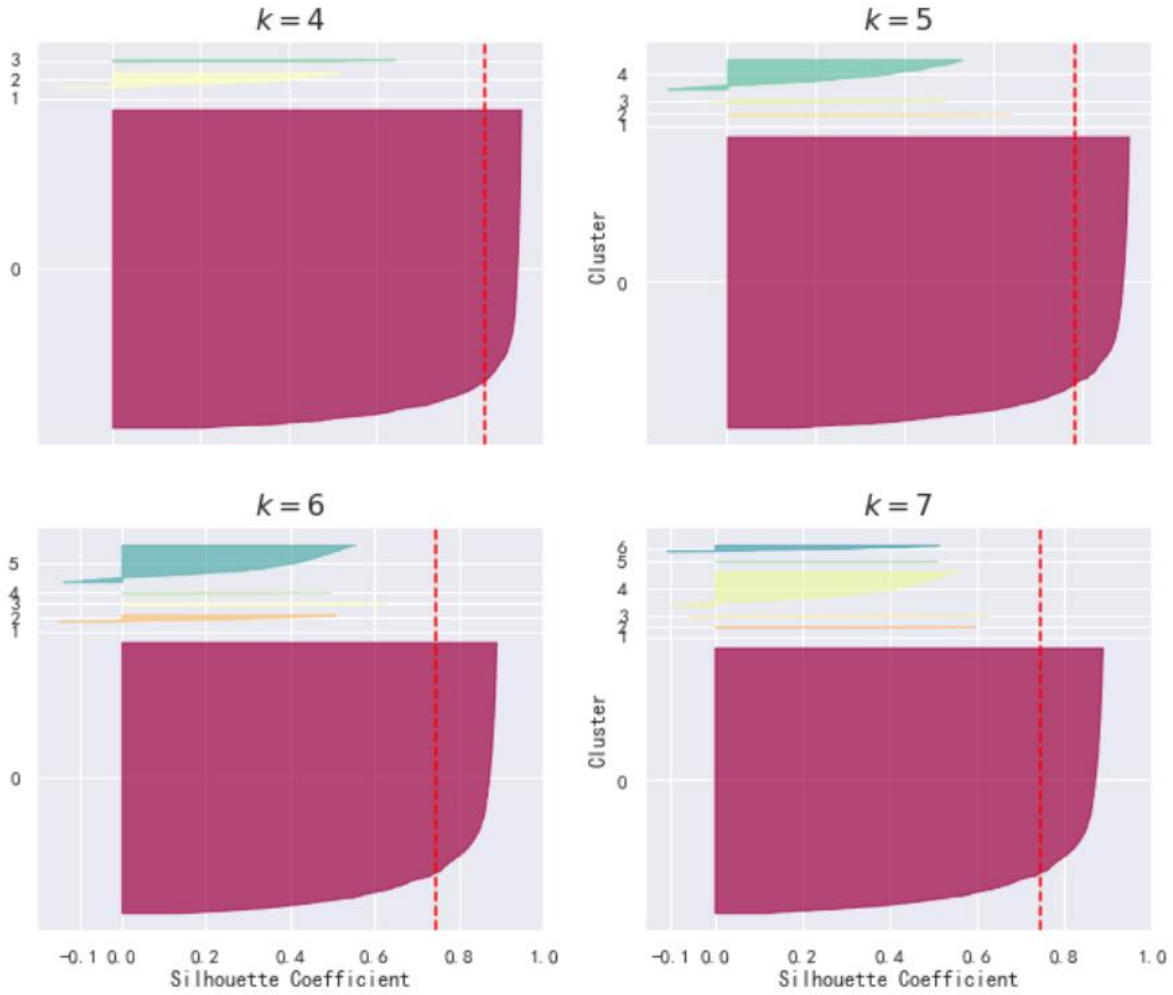


图 4-3 聚类轮廓图

每个轮廓图包含一个刀形的聚类，高度表示集群包含的实例数，宽度表示集群中实例的排序轮廓系数（越宽越好）。虚线表示平均轮廓系数。

垂直虚线表示每个集群的轮廓分数。当集群中的大多数实例的系数均低于此分数时（如果许多实例在虚线附近停止，在其左侧结束），则该集群比较糟糕，因为这意味着其实例太接近其他集群了。可以看到，没有特别好的集群，但相比而言使用 k=4 来获得相似大小的集群结果更优。

聚成 4 类后每一类的类中心为：

表 4-3 聚类类中心

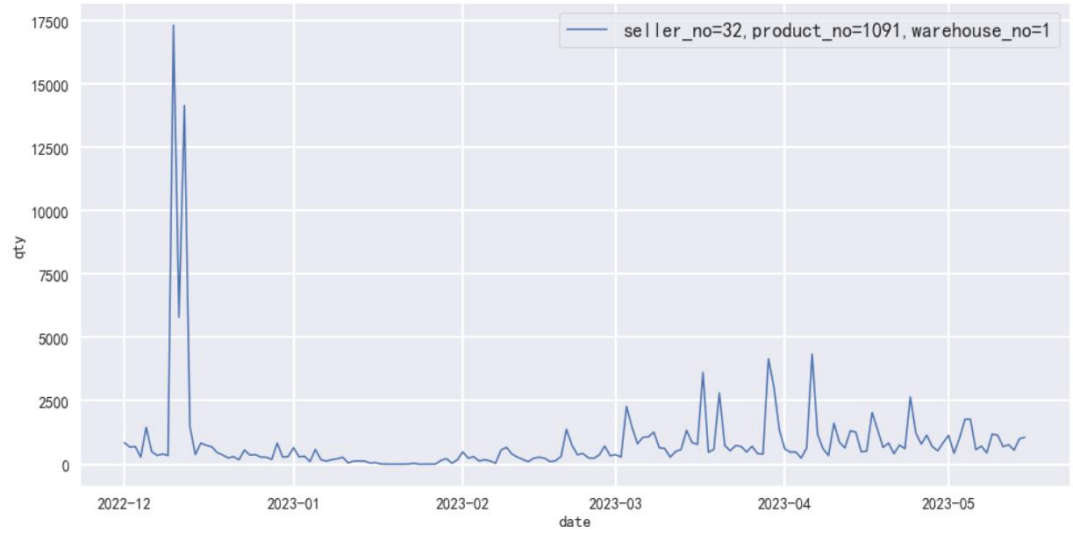
	mean	std	min	25%	50%	75%	max
cluster1	8.696274	6.595646	0.311346	4.151583	7.186016	11.586016	38.755145
cluster2	873.542169	1842.027084	0.000000	247.750000	485.000000	823.000000	17323.000000
cluster3	89.068634	65.283288	5.550562	45.182584	75.033708	114.483146	400.617978
cluster4	307.948521	311.734804	25.272727	143.295455	236.454545	387.431818	2300.909091

通过对输出的聚类类中心表数据进行分析，可以发现类与类之间仍有较大差距。

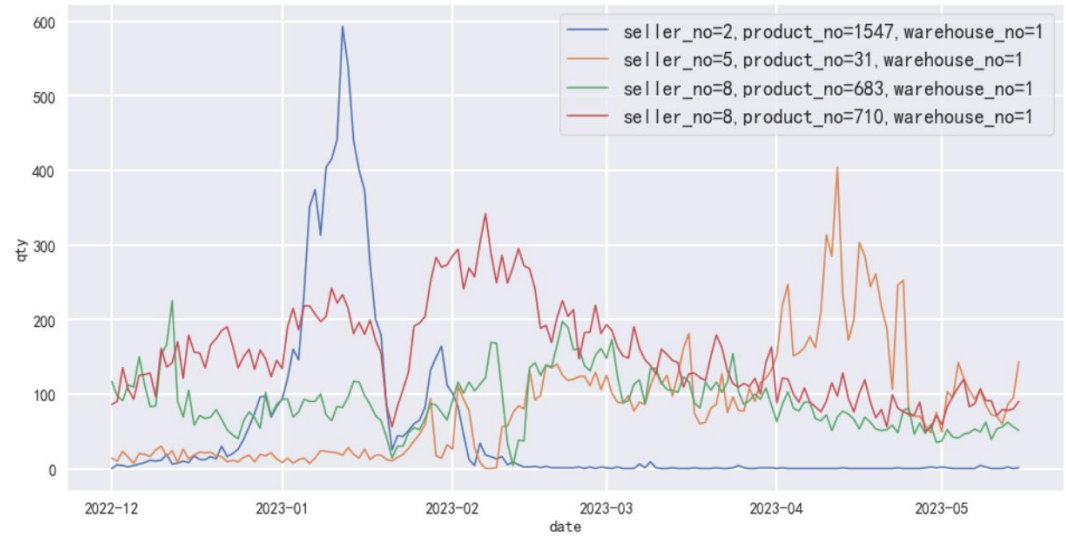
接着对每个类选择前 4 个组别，绘制出销售量随时间变化折线图，观察各个组别销售量数据的变化趋势，从而为预测模型的选择提供参考依据。



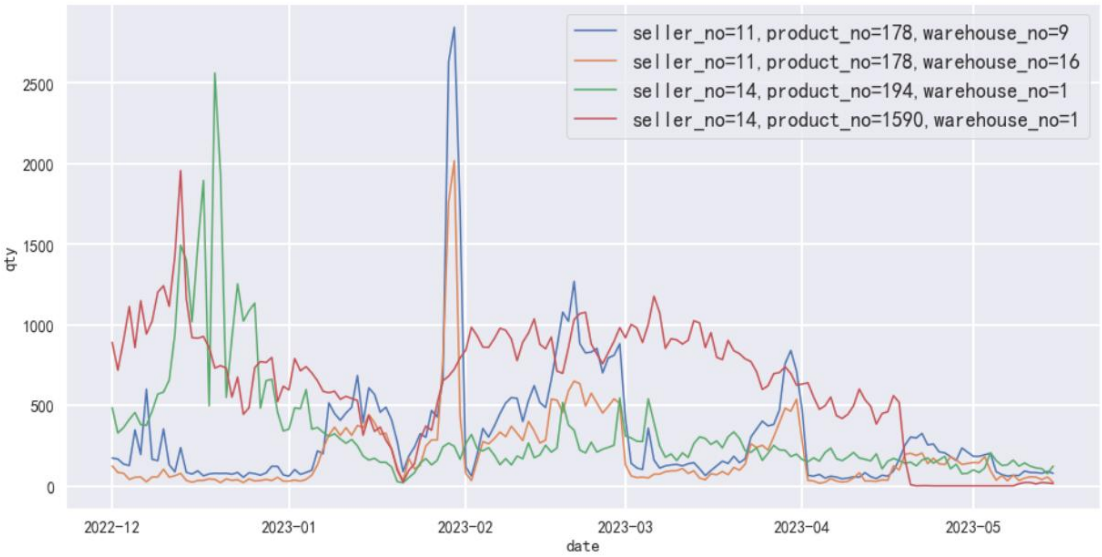
图（a） cluster1 前 4 组数据销售量随时间变化折线图



图（b） cluster2 前 4 组数据销售量随时间变化折线图



图（c） cluster3 前 4 组数据销售量随时间变化折线图



图（d） cluster4 前 4 组数据销售量随时间变化折线图

图 4-4 销售量随时间变化折线图

4.2.3 时间序列预测模型建立

预测模型一般有回归预测，微分方程，灰色预测，深度学习（如 RNN，LSTM 等），时间序列预测等，结合上述问题研究，发现销售量数据随时间**波动较大**，并考虑**模型的简易性与适用条件**，本文选择时间序列预测模型。

按序列统计特性划分，可以将时间序列划分为平稳时间序列，非平稳时间序列；如果一个时间序列的概率分布与时间 t 无关，则该时间序列为狭义平稳的，如果序列的一二阶矩存在，而且对任意时刻 t，均值为常数且协方差为时间间隔 τ 的函数，则该序列为广义平稳的[3]。

建立差分自回归移动平均模型(Auto Regressive Integrated Moving Average,ARIMA)：

p 阶 AR(p)：假设时间序列 x_t 仅与 $x_{t-1},...,x_{t-p}$ 有线性关系，而在 $x_{t-1},...,x_{t-p}$ 已知前提下，x_t 与 $x_{t-j}(j=p+1,...)$ 无关，a_t 是一个独立于 $x_{t-1},...,x_{t-p}$ 的白噪声序列，$a_t \sim N(0,\sigma_a^2)$。

$$x_t = \varphi_1 x_{t-1} + \varphi_2 x_{t-2} + ... + \varphi_p x_{t-p} + a_t \tag{4.5}$$

上式还可以表示为

$$a_t = x_t - \varphi_1 x_{t-1} - \varphi_2 x_{t-2} - ... - \varphi_p x_{t-p} \tag{4.6}$$

可见，AR(p)系统的响应 x_t 具有 p 阶动态性。 AR(p)模型通过把 x_t 中的依赖于 $x_{t-1},...,x_{t-p}$ 的部分消除掉之后，使得具有 p 阶动态性的序列 x_t 转化为独立的序列 a_t。因此，拟合 AR(p)模型的过程也就是使相关序列独立化的过程。

q 阶 MA(q)：AR(p)系统的特征是系统在 t 时刻的响应 x_t 仅与其以前时刻的响应 $x_{t-1},...,x_{t-p}$ 有关，而与其以前时刻进入系统的扰动无关。如果一个系统在 t 时刻的响应 x_t，与其以前时刻的响应 无关，而与其以前时刻进入系统的扰动 $a_{t-1},...,a_{t-q}$ 存在着一定的相关关系，那么，这一类系统为 MA(q)系统。

$$x_t = a_t - \theta_1 a_{t-1} - \theta_2 a_{t-2} - ... - \theta_q a_{t-p} \tag{4.7}$$

p,q 阶 **ARMA(p,q)**：一个系统，如果它在时刻 t 的响应 x_t，不仅与其以前时刻的自身值有关，而且还与其以前时刻进入系统的扰动存在一定的依存关系，那么，这个系统就是自回归移动平均系统。

$$x_t - \varphi_1 x_{t-1} - \varphi_2 x_{t-2} - ... - \varphi_p x_{t-p} = a_t - \theta_1 a_{t-1} - \theta_2 a_{t-2} - ... - \theta_q a_{t-p}$$

ARIMA 模型（Auto Regressive Integrated Moving Average model），差分整合自回归移动平均。自然界中绝大多数序列是非平稳的，可以通过差分运算将其变为平稳。

ARIMA(p,d,q)：

$$\Phi(B)\nabla^d x_t = \lambda + \Theta(B)\varepsilon_t \tag{4.8}$$

其中 B 为延迟算子，∇ 为差分运算，d 为使序列 x_t 成为平稳序列所做的差分次数，λ 为常数项，ε_t 为误差项

$$\begin{aligned} \nabla^d &= (1-B)^d \\ \Phi(B) &= 1 - \phi_1 B - ... - \phi_p B^p \\ \Theta(B) &= 1 - \theta_1 B - ... - \theta_q B^q \end{aligned} \tag{4.9}$$

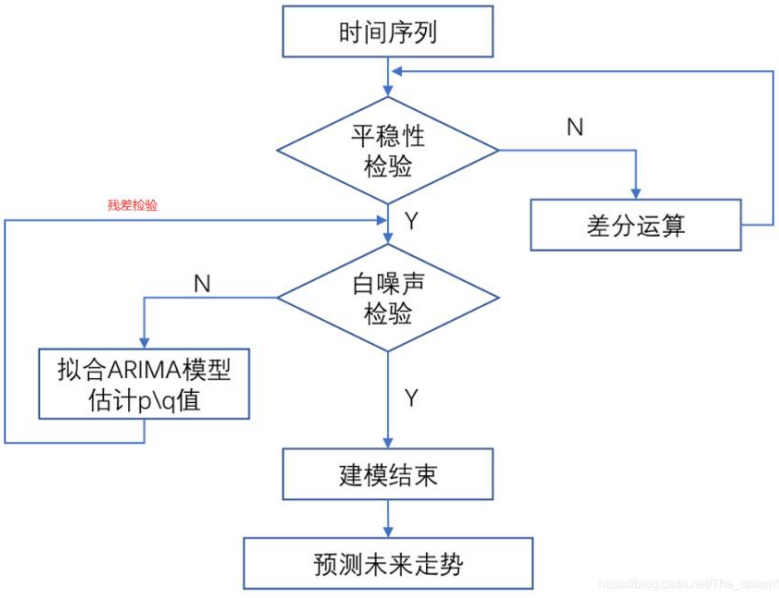


图 4-5 ARIMA 流程图

4.2.4 时间序列预测模型求解

以组合(seller_2,product_65,warehouse_1)为例子，进行模型求解展示：

（1）对序列进行平稳性检验

先画出原始数据的时序图，发现有比较大的起伏，再画出一次差分后数据的时序图，发现一次差分后数据趋于平稳。然后对序列进行 ADF 单位根检验，利用 python 函数 adfuller 进行检验，如果返回值 p<0.05, 则拒绝原假设，表示序列不存在单位根，说明数据平稳。如果原始时间序列不平稳，那就需要此时对原始数据取差分，再进行检验。先进行一阶差分，结果平稳就结束差分过程。如果依旧不平稳的话，再次求差分，直至通过检验。结果表明原始序列不是平稳序列，经过一次差分后的序列是平稳序列。

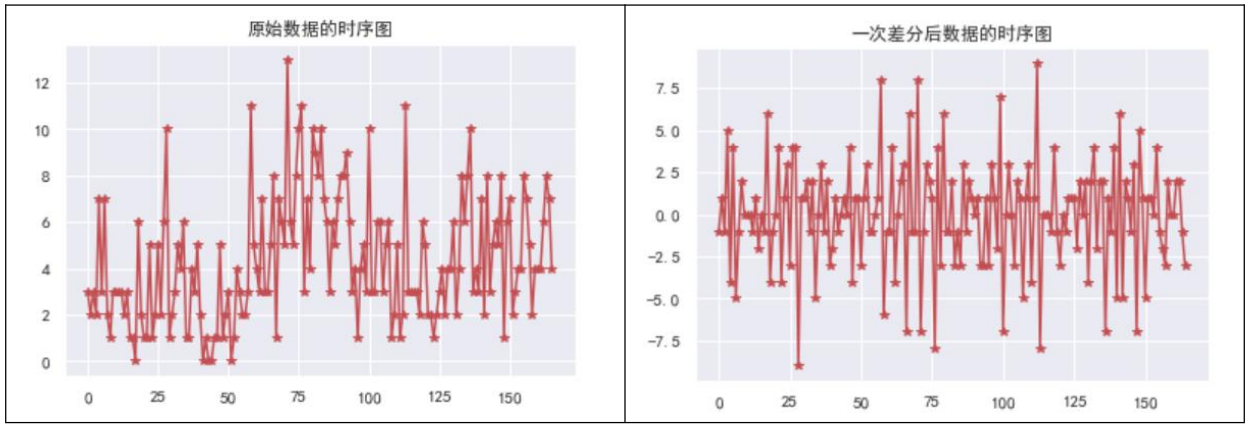


图 4-6 数据时序图

（2）对序列进行白噪声检验

考虑到白噪声是纯随机性数列，倘若对其建模则毫无意义，因此利用 python 中的 Ljung_Box 检验，观察延迟 6 期与 12 期后假设检验的 p 值，均是小于 0.05，故拒绝原假设，序列非白噪声。

（3）ARIMA(p,1,q)模型的定阶

通过赤池信息量准则（Akaike information criterion，简称 AIC）以及贝叶斯信息量准则（Bayesian information criterion，简称 BIC）来确定模型 ARIMA 的阶数 p,q。确定采用 ARIMA(1,1,2)。

时间序列模型参数估计值

变量	参数值	标准差	z 统计量	p 值
AR(1)	-1	0.016	-60.872	0
MA(1)	0.163	0.052	3.159	0.002
MA(2)	-0.837	0.052	-16.218	0

其中常数项在估计时已被剔除。

$$(1+B)(1-B)k_t=(1-0.163B+0.837B^2)\varepsilon_t$$

$$(1-B^2)k_t=(1-0.163B+0.837B^2)\varepsilon_t$$

$$k_t-k_{t-2}=\varepsilon_t-0.163\varepsilon_{t-1}+0.837\varepsilon_{t-2}$$

$$k_t=k_{t-2}+\varepsilon_t-0.163\varepsilon_{t-1}+0.837\varepsilon_{t-2} \quad (4.10)$$

另外，通过自相关图与偏自相关图看出，均表现拖尾的现象，因此选用 ARIMA 模型进行建模是合理的。

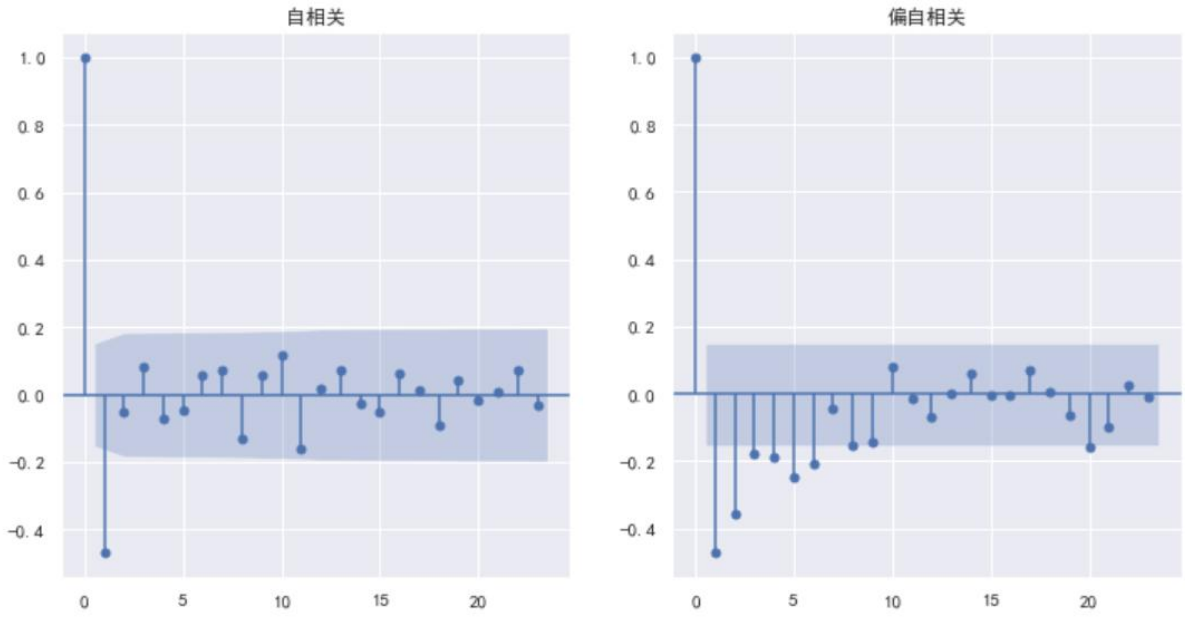


图 4-7 自相关图与偏自相关图

（4）模型的显著性检验/残差检验

一个好的拟合模型应该能够提取观察值序列中几乎所有的样本相关信息，即残差序列应该为白噪声序列。

反之，如果残差序列为非白噪声序列，那就意味着残差序列中还残留着相关信息未被提取，这就说明拟合模型不够有效。

残差图显示，残差大体满足均值为 0 的高斯分布，因此可认为显著性检验通过。

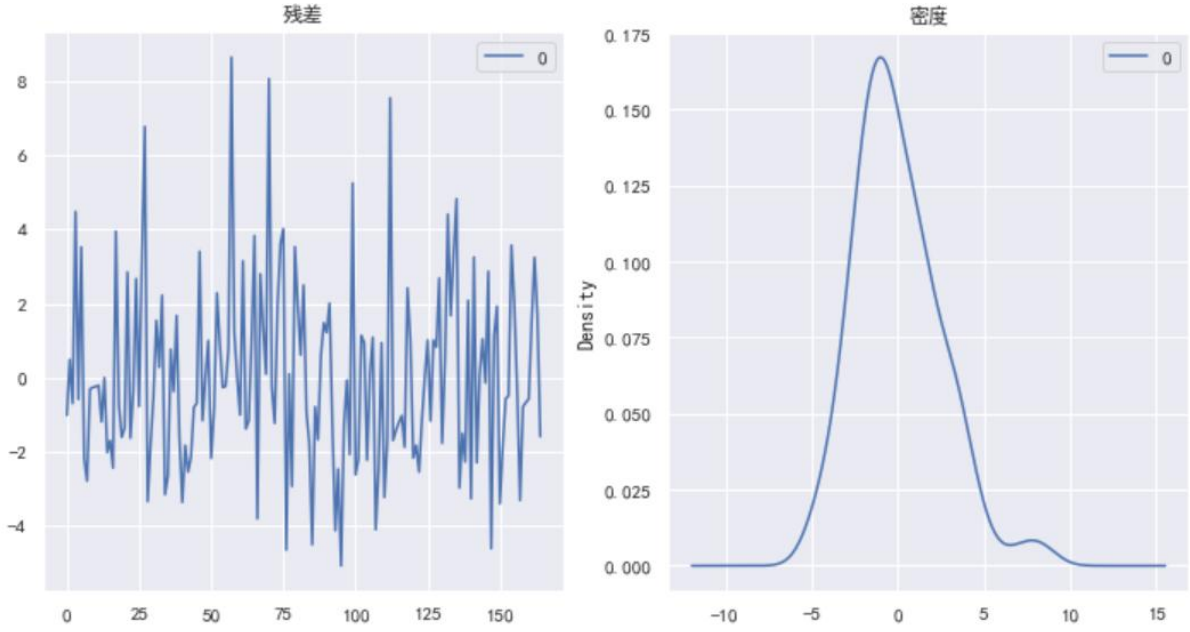


图 4-8 残差检验

（5）结果预测

首先给出预测模型评价指标 1-wmape：

$$1-wmape=1-\frac{\sum|y_i-\hat{y}_i|}{\sum y_i} \tag{4.11}$$

其中，y_i 为第 i 个序列（商家在各仓库中存放的各种商品每天的数量）的真实需求量，$\hat{y}_i$ 为第 i 个序列的预测需求量。

计算得到 1-wmape 为 0.5448。

销售量拟合图像如下图所示：

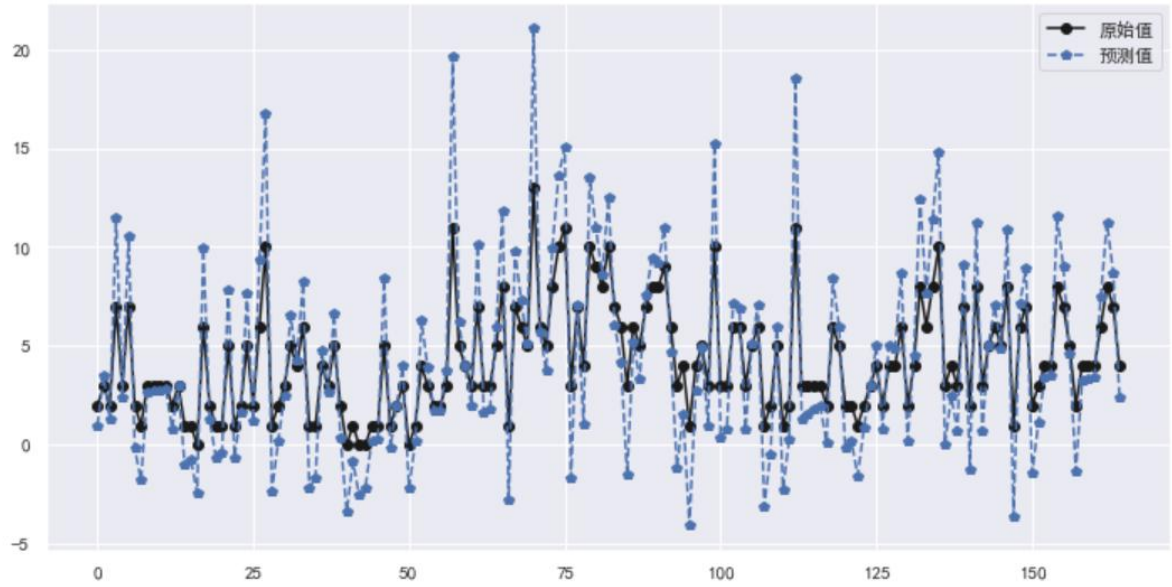


图 4-9 销售量拟合图像

可以看出，预测值虽然与真实值有一定差距，但趋势型规律是一致的。最后，本文对每个分组选择特定的 ARIMA 模型（四个分组一共选择四个），对 1996 个组合的销售量进行了 15 天的预测，预测结果见“结果表 1”。

4.3 模型小结和分析

本小问针对不同商家不同商品不同仓库的 1996 种组合，利用原始的 166 天销售量数据，基于销售量的统计特征（如最小值，最大值，中位数，上四分位数，下四分位数，平均值，标准差）进行聚类，然后针对每一类选取合适的 ARIMA 预测模型进行为期 15 天的销售量预测。

针对预测模型，本小问做了严格的模型检验，包括平稳性检验，白噪声检验与残差检验。相比于深度学习模型，建立的时间序列预测模型具有**简易性与适用性**。

5. 问题二的分析及模型建立与求解

5.1 问题分析与分解

问题二是针对附件 5 的数据，预测出各商家在各仓库的商品 2023-05-16 至 2023-05-30 的需求量。最重要的是，如何将这些新增的维度与附件 1 数据结合在一起，找到和附件 1 的相似序列，并参考问题 1 的解决方式更加准确地解决本问题。最后把预测结果填写在结果表 2。

问题二本质依然是一个**时间序列预测问题**，相比于问题一特点是组合数量只有 210 种，并且时间维度也不一样，历史数据有 36 天，28 天，45 天等等，如图 5-1 所示，可以利用问题一的模型，将 210 种组合和问题一的四个类别的类别中心作比较，选择最近的类别中心对应的类别，然后再利用问题一的预测模型进行销售量的预测。

本问题的核心和难点是：

针对 210 种组合（不同商家不同商品不同仓库），如何进行分类并对每一类选择合适的时间序列模型进行预测，来使得模型的精度更高。

问题 2 的求解分为如下几个关键步骤，求解流程图如下：

（1）首先数据提取，对附件 5 中的 210 种组合（不同商家不同商品不同仓库）进行销售量数据提取。

（2）然后进行分类，计算 210 种组合到问题 1 中 4 个类中心的距离，选取较小的距离对应的类。

（3）然后进行预测，利用问题 1 中不同的类对应的预测模型进行预测。

（4）最后对模型进行小结和分析。

			qty							
			count	mean	std	min	25%	50%	75%	max
seller_no1	product_no1	warehouse_no1								
1	3	5	36.0	0.361111	0.761682	0.0	0.00	0.0	0.00	3.0
	4	1	28.0	4.250000	6.714412	0.0	0.00	1.0	6.25	32.0
	8	3	36.0	2.694444	3.631760	0.0	0.75	1.0	2.50	12.0
	9	2	45.0	1.088889	1.239909	0.0	0.00	1.0	2.00	6.0
		5	45.0	1.155556	1.770408	0.0	0.00	1.0	2.00	8.0
...	...	...	...	...	...	...	...	...	...	...
35	2257	1	23.0	4.043478	3.948282	0.0	1.00	3.0	5.50	14.0
	2259	1	43.0	2.860465	1.958880	0.0	1.50	3.0	4.00	10.0
	2260	1	36.0	12.666667	12.457929	0.0	4.00	9.0	19.75	60.0
	2356	24	44.0	0.409091	1.063522	0.0	0.00	0.0	0.00	6.0
	2358	53	44.0	1.863636	2.257709	0.0	0.00	1.0	3.00	10.0

图 5-1 附件 5 商品 210 种组合

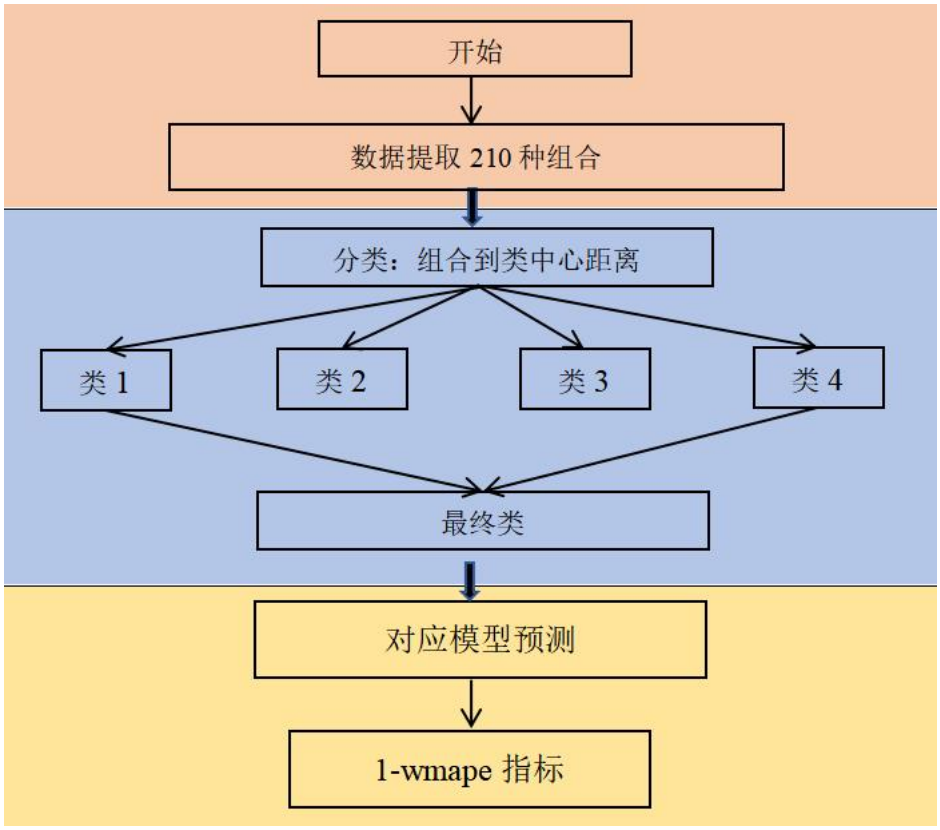


图 5-2 问题 2 技术路线图

5.2 模型建立

计算 210 种组合到各类中心之间的欧式距离，选取距离最小的类，将组合归为该类，120 种组合一共归为了两类，分别是第一类和第三类，数量分别为 207 和 3。

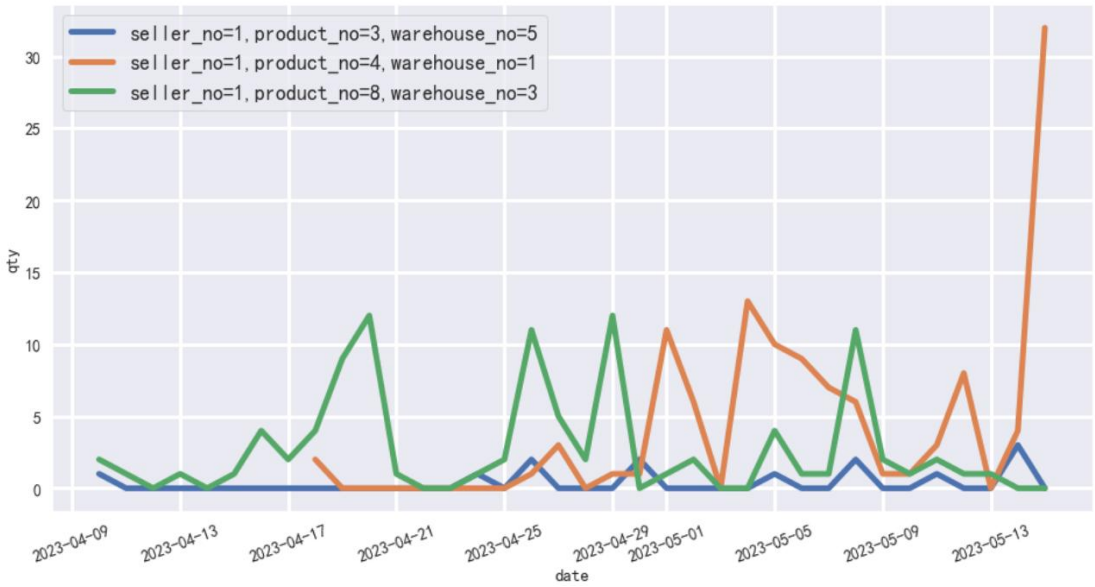
210 种组合归类示例数据如下表所示：

表 5-1 210 种组合归类示例数据

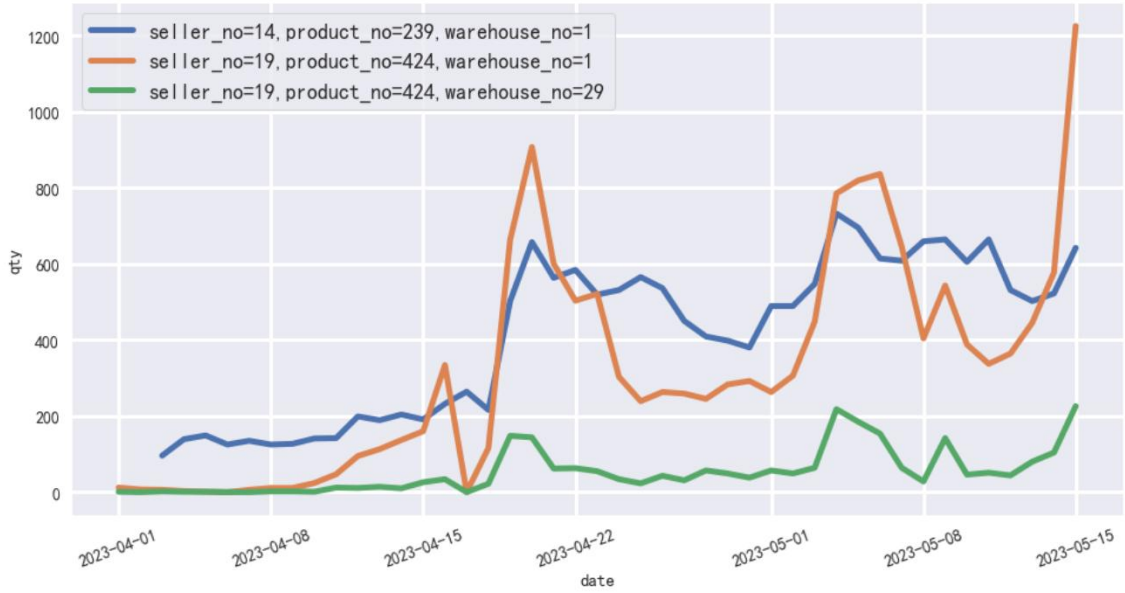
mean	std	min	25%	50%	75%	max	distance1	distance2	distance3	distance4	cluster
0.361111111	0.761681515	0	0	0	0	3	39.81391283	17467.37065	436.9677804	2387.234164	0
4.25	6.714412359	0	0	1	6.25	32	12.22652368	17437.47227	407.1503478	2356.932204	0
2.694444444	3.631760254	0	0.75	1	2.5	12	29.88535678	17457.87156	426.9796507	2377.347604	0
1.088888889	1.239908762	0	0	1	2	6	36.15171968	17464.1873	433.325583	2383.767287	0
1.155555556	1.770407703	0	0	1	2	8	34.25661082	17462.14492	431.4130205	2381.764215	0
1.111111111	1.885618083	0	0	0	2	9	33.55519075	17461.17117	430.668342	2380.891373	0
0.418604651	0.763218869	0	0	0	1	3	39.52217807	17467.32053	436.6949502	2387.064466	0
0.785714286	1.257780021	0	0	0	1	5	37.5701308	17465.26692	434.7269029	2385.02749	0

0.454545455	0.61699418	0	0	0	1	2	40.443123	17468.32571	437.619966	2388.04155	0
2.242424242	2.9050635	0	0	1	3	15	27.36601063	17454.98305	424.3970012	2374.576038	0

对于每个类通过选择前 3 个组别的方式，绘制出销售量随时间变化折线图，观察各个组别销售量数据的变化趋势：



图（a） 类 1 前 3 组数据销售量随时间变化折线图



图（b） 类 3 前 3 组数据销售量随时间变化折线图

图 5-3 销售量随时间变化折线图

5.3 模型求解

类别 1 和类别 3 使用的模型为：

$$k_t = k_{t-2} + \varepsilon_t - 0.163\varepsilon_{t-1} + 0.837\varepsilon_{t-2} \tag{5.1}$$

$$k_t = k_{t-1} + k_{t-2} + \varepsilon_t - 0.1\varepsilon_{t-1} + 0.267\varepsilon_{t-2} \tag{5.2}$$

以组合(seller_1,product_4,warehouse_1)为例子，进行模型求解展示：

表 5-2 销售量预测结果

seller_no	product_no	warehouse_no	date	forecast_qty
seller_1	product_4	warehouse_1	2023-05-16	17.847476874937705
seller_1	product_4	warehouse_1	2023-05-17	6.733319247267341
seller_1	product_4	warehouse_1	2023-05-18	12.33794174964002
seller_1	product_4	warehouse_1	2023-05-19	10.467627946544109
seller_1	product_4	warehouse_1	2023-05-20	11.939344622717893
seller_1	product_4	warehouse_1	2023-05-21	11.916845526618625
seller_1	product_4	warehouse_1	2023-05-22	12.56240759511934
seller_1	product_4	warehouse_1	2023-05-23	12.909280727304152
seller_1	product_4	warehouse_1	2023-05-24	13.389697141758553
seller_1	product_4	warehouse_1	2023-05-25	13.810406596660647
seller_1	product_4	warehouse_1	2023-05-26	14.257810925002978
seller_1	product_4	warehouse_1	2023-05-27	14.693280023795293
seller_1	product_4	warehouse_1	2023-05-28	15.134085342095466
seller_1	product_4	warehouse_1	2023-05-29	15.572504846321706
seller_1	product_4	warehouse_1	2023-05-30	16.011991043697844

拟合图像如下图所示：

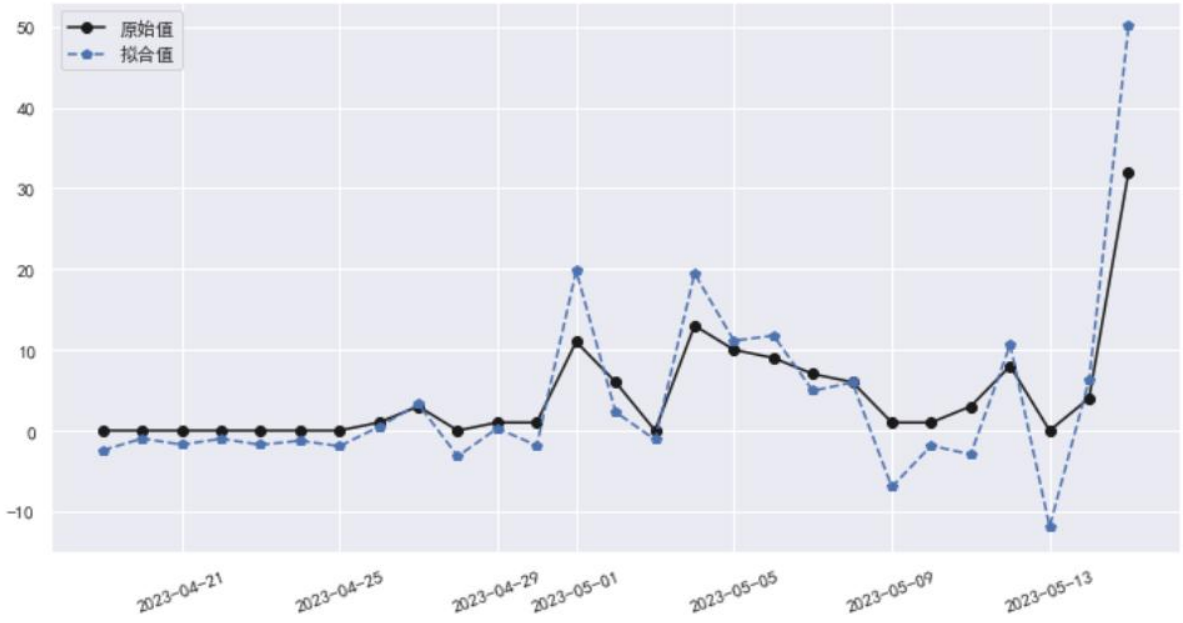


图 5-4 销售量拟合图像

可以看出拟合效果还比较好，拟合值虽然与真实值有一定差距，但趋势型规律是一致的。最后，通过对每个分组选择特定的 ARIMA 模型（两个分组一共选择两个），对 210 个组合的销售量进行了 15 天的预测，预测结果见“结果表 2”。

5.4 模型小结和分析

本小问针对不同商家不同商品不同仓库的 210 种组合，利用原始的不同维度销售量数据，并基于问题 1 的模型，计算 210 种组合和问题 1 的 4 个类别中心之间的距离，选择最近的类别中心对应的类别，然后针对每一类选取合适的 ARIMA 预测模型进行为期 15 天的销售量预测。

6．问题三的分析及模型建立与求解

6.1 问题分析与分解

问题三是针对附件 6 的数据，附件 6 给出了附件 1 对应的商家+仓库+商品维度在去年双十一期间的需求量数据，基于这些数据，需要给出 2023-06-01 到 2023-06-20 的预测值。并将预测结果填写在结果表 3。

问题三本质依然是一个**时间序列预测问题**，组合数量有 1957 种，时间维度大多数是 11 天，如图 6-1 所示，由于在问题一中，本文已经成功预测出了 2023-05-16 到 2023-05-30 的销售量，但是由于预测值本身存在误差，再作为原始数据来预测 6 月的数据就会产生误差叠加的后果。因此本文只选用 11 个历史数据来预测未来 9 天的销售量，由此得到 2022-06-01 到 2022-06-20 的销售量数据，然后再基于平均残差进行调整，得到 2023-06-01 到 2023-06-20 的销售量数据。

本问题的核心和难点是：

针对 1957 种组合（不同商家不同商品不同仓库），如何进行分类并对每一类选择合适的时间序列模型进行预测，来使得模型的精度更高。

问题 3 的求解分为如下几个关键步骤，求解流程图如下：

（1）首先数据提取，对附件 6 中的 1957 种组合（不同商家不同商品不同仓库）进行销售量数据提取。

（2）然后聚类预测，如问题一。

（3）然后根据残差调整，根据平均残差对 2022 年的数据调整得到 2023 年的预测数据。

（4）最后对模型进行小结和分析。

			qty							
			count	mean	std	min	25%	50%	75%	max
seller_no1	product_no1	warehouse_no1								
2	65	1	11.0	2.818182	1.834022	0.0	1.5	2.0	4.5	5.0
	66	1	11.0	3.363636	2.693426	0.0	1.5	3.0	4.5	8.0
	67	1	11.0	3.818182	2.442056	0.0	2.5	4.0	5.0	8.0
	69	1	11.0	3.363636	2.062655	2.0	2.0	3.0	3.0	9.0
		4	11.0	0.545455	0.522233	0.0	0.0	1.0	1.0	1.0
...	...	...	...	...	...	...	...	...	...	...
37	1461	8	11.0	2.363636	3.264130	0.0	0.0	1.0	2.5	10.0
	1462	8	11.0	3.000000	3.847077	0.0	0.0	2.0	4.0	11.0
	1463	37	11.0	0.090909	0.301511	0.0	0.0	0.0	0.0	1.0
	1464	10	11.0	0.818182	1.250454	0.0	0.0	0.0	1.5	3.0
	1465	10	11.0	19.454545	8.358991	8.0	14.5	19.0	22.0	36.0

图 6-1 附件 6 商品 1957 种组合

6.2 模型建立

首先绘制出前 4 种组合的销售量随时间变化折线图，看是否能找到一些规律：

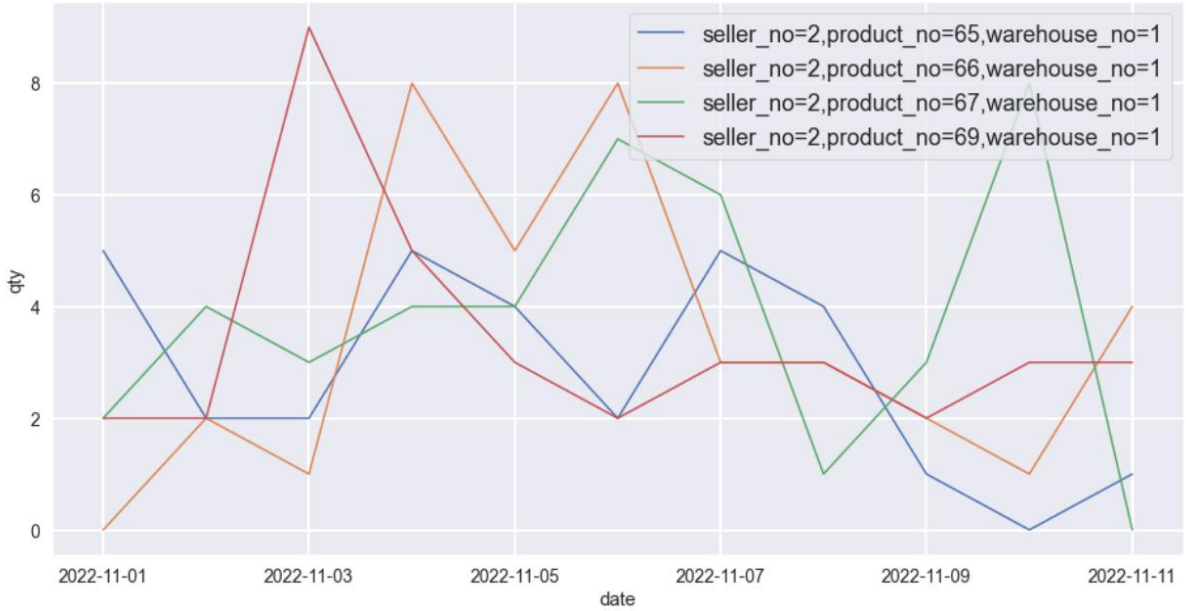


图 6-2 前 4 组数据销售量随时间变化折线图

发现并没有规律，然后利用 k-均值重新对附件 6 数据进行聚类。

将 1957 种组合分别聚成 4，5，6，7 类的轮廓系数为：**0.8570**，0.8569，0.8467，0.7641，绘制出轮廓图：

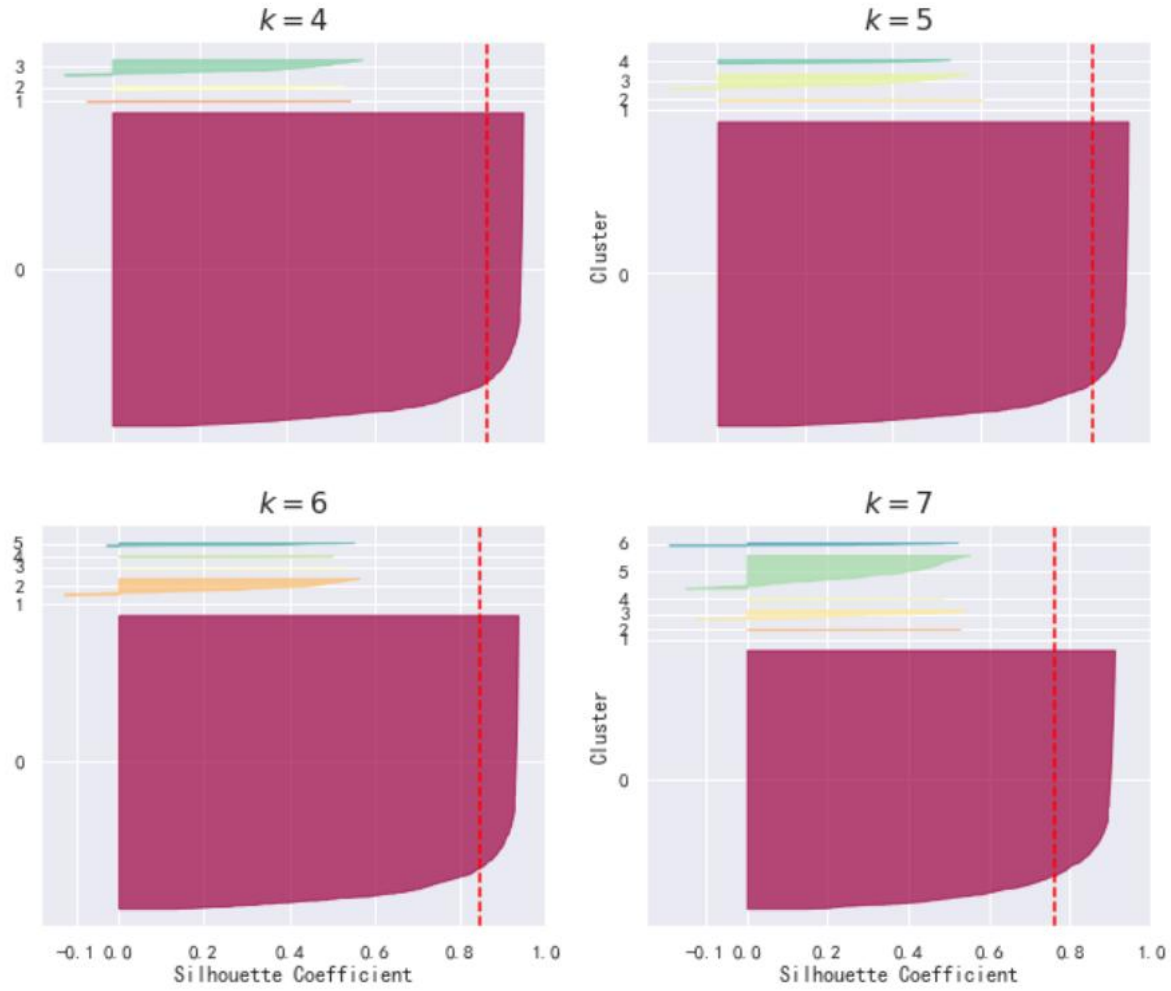


图 6-3 聚类轮廓图

聚成 4 类后每一类的类中心为：

表 6-1 聚类类中心

	mean	std	min	25%	50%	75%	max
cluster1	12.961613	8.535188	4.276480	7.583379	10.631722	15.606871	31.741445
cluster2	1885.204545	2188.874625	65.000000	433.250000	925.000000	2462.125000	6980.750000
cluster3	526.966942	513.002015	111.409091	216.818182	351.500000	607.909091	1781.727273
cluster4	153.708081	141.476993	40.566667	69.872222	106.200000	169.888889	502.811111

通过对表 6-1 聚类类中心的数据结果进行分析，可以得出类与类之间仍有较大差别的结果，本文接下来将为每个类选择不同的时间序列模型进行预测。

6.3 模型求解

以组合(seller_2,product_66,warehouse_1)为例子，进行模型求解展示：

表 6-2 销售量预测结果

seller_no	product_no	warehouse_no	date	forecast_qty
seller_2	product_66	warehouse_1	2023-06-1	0.132168344
seller_2	product_66	warehouse_1	2023-06-2	2.132168344
seller_2	product_66	warehouse_1	2023-06-3	1.132168344
seller_2	product_66	warehouse_1	2023-06-4	8.132168344

seller_2	product_66	warehouse_1	2023-06-5	5.132168344
seller_2	product_66	warehouse_1	2023-06-6	8.132168344
seller_2	product_66	warehouse_1	2023-06-7	3.132168344
seller_2	product_66	warehouse_1	2023-06-8	3.132168344
seller_2	product_66	warehouse_1	2023-06-9	2.132168344
seller_2	product_66	warehouse_1	2023-06-10	1.132168344
seller_2	product_66	warehouse_1	2023-06-11	4.132168344
seller_2	product_66	warehouse_1	2023-06-12	3.578443157
seller_2	product_66	warehouse_1	2023-06-13	3.810831908
seller_2	product_66	warehouse_1	2023-06-14	4.04322066
seller_2	product_66	warehouse_1	2023-06-15	4.275609411
seller_2	product_66	warehouse_1	2023-06-16	4.507998163
seller_2	product_66	warehouse_1	2023-06-17	4.740386914
seller_2	product_66	warehouse_1	2023-06-18	4.972775666
seller_2	product_66	warehouse_1	2023-06-19	5.205164417
seller_2	product_66	warehouse_1	2023-06-20	5.437553169

最后，本文对每个分组选择特定的 ARIMA 模型（四个分组一共选择四个），对 1957 个组合的销售量进行了 20 天的预测，预测结果可见“结果表 3”。

6.4 模型小结和分析

本小问针对不同商家不同商品不同仓库的 1957 种组合，利用双 11 期间的 11 天原始销售量数据，并基于问题 1 的方法，针对每一类选取合适的 ARIMA 预测模型进行为期 9 天的销售量预测，然后基于预测模型的平均残差对 2022 年的 20 天数据进行调整，得到 2023-06-01 到 2023-06-20 的预测值。

7．模型的评价与改进

7.1 模型优点

1.紧扣数据分布特征，模型思想相互联系：

本文对问题的研究由浅入深，建立时间序列预测模型，运用类似的模型思想解决不同实际问题。

针对问题一，对于商品分类情况下的需求量预测，针对不同商家不同商品不同仓库的 1996 种组合，利用原始的 166 天销售量数据，基于销售量的统计特征（如最小值，最大值，中位数，上四分位数，下四分位数，平均值，标准差）进行聚类，然后针对每一类选取合适的 ARIMA 预测模型进行为期 15 天的销售量预测。

针对问题二，对于增维度下的商品需求量预测，一共是 210 种组合，利用原始的不同维度销售量数据，并基于问题 1 的模型，计算 210 种组合和问题 1 的 4 个类别中心之间的距离，选择最近的类别中心对应的类别，然后针对每一类选取合适的 ARIMA 预测模型进行为期 15 天的销售量预测。

针对问题三，对于大型促销货物货量陡增情况下的商品需求量预测，是有 1957 种组合，利用双 11 期间的 11 天原始销售量数据，并基于问题 1 的方法，针对每一类选取合适的 ARIMA 预测模型进行为期 9 天的销售量预测，然后基于预测模型的平均残差对

2022 年的 20 天数据进行调整，得到 2023-06-01 到 2023-06-20 的预测值。

2.对预测模型进行验证，验证其合理可靠：

在求解本题过程中，针对预测模型，做出严格的模型检验，包括平稳性检验，白噪声检验与残差检验，验证采用该模型的合理性。相比于深度学习模型，建立的时间序列预测模型具有简易性与适用性。更进一步验证了实验结果的可靠性，说明使用模型普遍性强，吻合率高。

7.2 模型缺点与改进

1. 建模方法比较局限，没有尝试更多模型：

对于问题的研究，本文主要采用时间序列模型 ARIMA，在方法应用上比较单调，未来可以尝试深度学习模型，尝试引入更多复杂和灵活的建模方法，并寻找模型的可解释性。

参考文献

[1] 周志华,《机器学习》,北京:清华大学出版社,2016.

[2] 技术宅 zch,聚类效果评估——轮廓系数（Silhouette Coefficient）附 Python 代码,https://blog.csdn.net/qq_19672707/article/details/106857918,2023 年 11 月 1 日.

[3] 王燕,《应用时间序列分析》（第五版）,北京:中国人民大学出版社,2019.

附录

问题一, 二，三 python 程序代码：main.ipynb，这里仅展示问题一部分代码，全部代码见附件。

```
# 数据分析所需的库
import numpy as np
import pandas as pd
import re

# 数据可视化所需的库
import matplotlib.pyplot as plt
from matplotlib.ticker import FixedLocator, FixedFormatter
import matplotlib as mpl
plt.rcParams['font.sans-serif']='SimHei'
plt.rcParams['axes.unicode_minus']=False
import seaborn as sns

# k 均值聚类所需的库
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score          # 样本平均轮廓系数
from sklearn.metrics import silhouette_samples       # 所有样本的轮廓系数

# 时间序列所需的库
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller
from statsmodels.stats.diagnostic import acorr_ljungbox as lb_test
import statsmodels.api as sm

# 其他所需的库
import warnings
warnings.filterwarnings('ignore')


# 读取附件 1-附件 4 数据并按顺序排列
## 提取字符串中的数字
def extract_numbers(input_string):
    return re.findall('\d+', input_string)
## 读取附件 1-商家历史出货量表.xlsx
table1 = pd.read_excel(r'./datasets/附件 1-商家历史出货量表.xlsx')
table1['seller_no1'] = [int(extract_numbers(i)[0]) for i in table1['seller_no']]
table1['product_no1'] = [int(extract_numbers(i)[0]) for i in table1['product_no']]
table1['warehouse_no1'] = [int(extract_numbers(i)[0]) for i in table1['warehouse_no']]
table1.sort_values(by=['seller_no1', 'product_no1', 'warehouse_no1', 'date'], inplace=True)
table1
## 读取附件 2-商品信息表.xlsx
table2 = pd.read_excel(r'./datasets/附件 2-商品信息表.xlsx')
```

```
table2['product_no1'] = [int(extract_numbers(i)[0]) for i in table2['product_no']]
table2.sort_values(by='product_no1', inplace=True)
table2
## 读取附件 3-商家信息表.xlsx
table3 = pd.read_excel(r'./datasets/附件 3-商家信息表.xlsx')
table3
## 读取附件 4-仓库信息表.xlsx
table4 = pd.read_excel(r'./datasets/附件 4-仓库信息表.xlsx')
table4['warehouse_no1'] = [int(extract_numbers(i)[0]) for i in table4['warehouse_no']]
table4.sort_values(by='warehouse_no1', inplace=True)
table4
## 读取附件 5-新品历史出货量表.xlsx
table5 = pd.read_excel(r'./datasets/附件 5-新品历史出货量表.xlsx')
table5['seller_no1'] = [int(extract_numbers(i)[0]) for i in table5['seller_no']]
table5['product_no1'] = [int(extract_numbers(i)[0]) for i in table5['product_no']]
table5['warehouse_no1'] = [int(extract_numbers(i)[0]) for i in table5['warehouse_no']]
table5.sort_values(by=['seller_no1', 'product_no1', 'warehouse_no1', 'date'], inplace=True)
table5
## 读取附件 6-促销期间商家出货量表.xlsx
table6 = pd.read_excel(r'./datasets/附件 6-促销期间商家出货量表.xlsx')
table6['seller_no1'] = [int(extract_numbers(i)[0]) for i in table6['seller_no']]
table6['product_no1'] = [int(extract_numbers(i)[0]) for i in table6['product_no']]
table6['warehouse_no1'] = [int(extract_numbers(i)[0]) for i in table6['warehouse_no']]
table6.sort_values(by=['seller_no1', 'product_no1', 'warehouse_no1', 'date'], inplace=True)
table6


# 数据预处理
## 删除重复值
print(table1.drop_duplicates(keep='first').shape)
print(table2.drop_duplicates(keep='first').shape)
print(table3.drop_duplicates(keep='first').shape)
print(table4.drop_duplicates(keep='first').shape)
print(table5.drop_duplicates(keep='first').shape)
print(table6.drop_duplicates(keep='first').shape)
## 填补缺失值
print(table1.isnull().sum())
print(table2.isnull().sum())
print(table3.isnull().sum())
print(table4.isnull().sum())
print(table5.isnull().sum())
print(table6.isnull().sum())
## 修改异常值
### 附件 1
temp = table1[['seller_no1', 'product_no1', 'warehouse_no1', 'date', 'qty']]
group1 = temp.groupby(by=['seller_no1', 'product_no1', 'warehouse_no1'])
```

```
group1.describe()
fig = plt.figure(1, (4, 2), dpi=250)
plt.subplot(111)
sns.set(style="darkgrid")
tips=pd.DataFrame(data=np.array([temp.iloc[:165*4,:]['qty'],[1]*165+[2]*165+[3]*165+[4
]*165]).T,
                          columns=['qty', ' '])
ax = sns.boxplot(x=" ", y="qty", data=tips)
plt.legend(['seller_no=2,product_no=65,warehouse_no=1','seller_no=2,product_no=66,war
ehouse_no=1',
                    'seller_no=2,product_no=67,warehouse_no=1',
'seller_no=2,product_no=69,warehouse_no=1'], fontsize=4)
temp1=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==65) &
(temp['warehouse_no1']==1)]
temp2=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==66) &
(temp['warehouse_no1']==1)]
temp3=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==67) &
(temp['warehouse_no1']==1)]
temp4=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==69) &
(temp['warehouse_no1']==1)]
fig = plt.figure(figsize=(16,8))
plt.plot(temp1['date'], temp1['qty'], label='seller_no=2,product_no=65,warehouse_no=1')
plt.plot(temp2['date'], temp2['qty'], label='seller_no=2,product_no=66,warehouse_no=1')
plt.plot(temp3['date'], temp3['qty'], label='seller_no=2,product_no=67,warehouse_no=1')
plt.plot(temp4['date'], temp4['qty'], label='seller_no=2,product_no=69,warehouse_no=1')
plt.xlabel('date', fontsize=15)
plt.ylabel('qty', fontsize=15)
# 修改 x 轴和 y 轴刻度的字体大小
plt.tick_params(axis='both', labelsize=14)
plt.grid(True, linewidth=2)
plt.legend(fontsize=18)
plt.show()
### 附件 6
temp = table6[['seller_no1', 'product_no1', 'warehouse_no1', 'date', 'qty']]
group6 = temp.groupby(by=['seller_no1', 'product_no1', 'warehouse_no1'])
group6.describe()
fig = plt.figure(1, (4, 2), dpi=250)
plt.subplot(111)
sns.set(style="darkgrid")
tips=pd.DataFrame(data=np.array([temp.iloc[:165*4,:]['qty'],[1]*165+[2]*165+[3]*165+[4
]*165]).T,
                          columns=['qty', ' '])
ax = sns.boxplot(x=" ", y="qty", data=tips)
plt.legend(['seller_no=2,product_no=65,warehouse_no=1','seller_no=2,product_no=66,war
ehouse_no=1',
                    'seller_no=2,product_no=67,warehouse_no=1',
```

```
'seller_no=2,product_no=69,warehouse_no=1'], fontsize=4)
temp1=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==65) &
(temp['warehouse_no1']==1)]
temp2=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==66) &
(temp['warehouse_no1']==1)]
temp3=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==67) &
(temp['warehouse_no1']==1)]
temp4=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==69) &
(temp['warehouse_no1']==1)]
fig = plt.figure(figsize=(16,8))
plt.plot(temp1['date'], temp1['qty'], label='seller_no=2,product_no=65,warehouse_no=1')
plt.plot(temp2['date'], temp2['qty'], label='seller_no=2,product_no=66,warehouse_no=1')
plt.plot(temp3['date'], temp3['qty'], label='seller_no=2,product_no=67,warehouse_no=1')
plt.plot(temp4['date'], temp4['qty'], label='seller_no=2,product_no=69,warehouse_no=1')
plt.xlabel('date', fontsize=15)
plt.ylabel('qty', fontsize=15)
# 修改 x 轴和 y 轴刻度的字体大小
plt.tick_params(axis='both', labelsize=14)
plt.grid(True, linewidth=2)
plt.legend(fontsize=18)
plt.show()


# 问题一模型求解
## 数据提取
# 1996 种组合
index = table1[['seller_no1', 'product_no1', 'warehouse_no1']]
index = index.drop_duplicates(keep='first')
Index
# 以提取第一个组合为例子，然后进行预测
a = table1.loc[(table1['seller_no1']==index.iloc[0,0]) &
                (table1['product_no1']==index.iloc[0,1]) &
                (table1['warehouse_no1']==index.iloc[0,2])]
qty = a['qty']
qty
## k-均值
column = group1.describe().columns
# 数量，平均值，标准差, 最小值，下四分位数，中位数，上四分位数，最大值
X = group1.describe()[column[1:]].values
X = pd.DataFrame(columns=column[1:],data=X)
X
model = [KMeans(
                n_clusters=i,
                # good_init = np.array([[-3, 3], [-3, 2], [-3, 1], [-1, 2], [0, 2]])
                # kmeans = KMeans(n_clusters=5, init=good_init, n_init=1)
                # 初始类中心设置,默认智能化选择
```

```
                    init='k-means++',
                    # 获取初始簇中心的迭代次数，为了弥补初始质心的影响，算法默认会初始 10 次质心，实现算法，然后返回最优的结果
                    n_init=10,
                    random_state=11
                ) for i in range(4,8)]
train_model = [i.fit(X) for i in model]
silhouette_scores = [silhouette_score(X, i.labels_) for i in train_model]
silhouette_scores
plt.figure(figsize=(11, 9))
for k in (4, 5, 6, 7):
    plt.subplot(2, 2, k - 3)

    y_pred = train_model[k - 4].labels_
    # 默认采用欧式聚类
    silhouette_coefficients = silhouette_samples(X, y_pred, metric='euclidean')

    padding = len(X) // 30
    pos = padding
    ticks = []
    for i in range(k):
        coeffs = silhouette_coefficients[y_pred == i]
        coeffs.sort()

        color = mpl.cm.Spectral(i / k)
        plt.fill_betweenx(np.arange(pos, pos + len(coeffs)), 0, coeffs,
                          facecolor=color, edgecolor=color, alpha=0.7)
        ticks.append(pos + len(coeffs) // 2)
        pos += len(coeffs) + padding

    plt.gca().yaxis.set_major_locator(FixedLocator(ticks))
    plt.gca().yaxis.set_major_formatter(FixedFormatter(range(k)))
    if k in (5, 7):
        plt.ylabel("Cluster")

    if k in (6, 7):
        plt.gca().set_xticks([-0.1, 0, 0.2, 0.4, 0.6, 0.8, 1])
        plt.xlabel("Silhouette Coefficient")
    else:
        plt.tick_params(labelbottom=False)

    plt.axvline(x=silhouette_scores[k - 4], color="red", linestyle="--")
    plt.title("$k={}$".format(k), fontsize=16)

plt.show()
model = KMeans(
```

```
                n_clusters=4,
                # good_init = np.array([[-3, 3], [-3, 2], [-3, 1], [-1, 2], [0, 2]])
                # kmeans = KMeans(n_clusters=5, init=good_init, n_init=1)
                #  初始类中心设置,默认智能化选择
                init='k-means++',
                #  获取初始簇中心的迭代次数，为了弥补初始质心的影响，算法默认会初始 10 次质心，实现算法，然后返回最优的结果
                n_init=10,
                random_state=11
            )
model.fit(X)
cluster_centers = pd.DataFrame(data=model.cluster_centers_ )
cluster_centers
#  类 2，1776
temp = table1[['seller_no1', 'product_no1', 'warehouse_no1', 'date', 'qty']]
temp1=temp.loc[(temp['seller_no1']==32) & (temp['product_no1']==1091) &
(temp['warehouse_no1']==1)]
fig = plt.figure(figsize=(16,8))
plt.plot(temp1['date'], temp1['qty'],
label='seller_no=32,product_no=1091,warehouse_no=1')
plt.xlabel('date', fontsize=15)
plt.ylabel('qty', fontsize=15)
#  修改 x 轴和 y 轴刻度的字体大小
plt.tick_params(axis='both', labelsize=14)
plt.grid(True, linewidth=2)
plt.legend(fontsize=18)
plt.show()
#  类 3，25，104，250，265
temp = table1[['seller_no1', 'product_no1', 'warehouse_no1', 'date', 'qty']]
temp1=temp.loc[(temp['seller_no1']==2) & (temp['product_no1']==1547) &
(temp['warehouse_no1']==1)]
temp2=temp.loc[(temp['seller_no1']==5) & (temp['product_no1']==31) &
(temp['warehouse_no1']==1)]
temp3=temp.loc[(temp['seller_no1']==8) & (temp['product_no1']==683) &
(temp['warehouse_no1']==1)]
temp4=temp.loc[(temp['seller_no1']==8) & (temp['product_no1']==710) &
(temp['warehouse_no1']==1)]
fig = plt.figure(figsize=(16,8))
plt.plot(temp1['date'], temp1['qty'], label='seller_no=2,product_no=1547,warehouse_no=1')
plt.plot(temp2['date'], temp2['qty'], label='seller_no=5,product_no=31,warehouse_no=1')
plt.plot(temp3['date'], temp3['qty'], label='seller_no=8,product_no=683,warehouse_no=1')
plt.plot(temp4['date'], temp4['qty'], label='seller_no=8,product_no=710,warehouse_no=1')
plt.xlabel('date', fontsize=15)
plt.ylabel('qty', fontsize=15)
#  修改 x 轴和 y 轴刻度的字体大小
plt.tick_params(axis='both', labelsize=14)
```

```
plt.grid(True, linewidth=2)
plt.legend(fontsize=18)
plt.show()
# 类 4，467，468，743，848
temp = table1[['seller_no1', 'product_no1', 'warehouse_no1', 'date', 'qty']]
temp1=temp.loc[(temp['seller_no1']==11) & (temp['product_no1']==178) &
(temp['warehouse_no1']==9)]
temp2=temp.loc[(temp['seller_no1']==11) & (temp['product_no1']==178) &
(temp['warehouse_no1']==16)]
temp3=temp.loc[(temp['seller_no1']==14) & (temp['product_no1']==194) &
(temp['warehouse_no1']==1)]
temp4=temp.loc[(temp['seller_no1']==14) & (temp['product_no1']==1590) &
(temp['warehouse_no1']==1)]
fig = plt.figure(figsize=(16,8))
plt.plot(temp1['date'], temp1['qty'], label='seller_no=11,product_no=178,warehouse_no=9')
plt.plot(temp2['date'], temp2['qty'],
label='seller_no=11,product_no=178,warehouse_no=16')
plt.plot(temp3['date'], temp3['qty'], label='seller_no=14,product_no=194,warehouse_no=1')
plt.plot(temp4['date'], temp4['qty'],
label='seller_no=14,product_no=1590,warehouse_no=1')
plt.xlabel('date', fontsize=15)
plt.ylabel('qty', fontsize=15)
# 修改 x 轴和 y 轴刻度的字体大小
plt.tick_params(axis='both', labelsize=14)
plt.grid(True, linewidth=2)
plt.legend(fontsize=18)
plt.show()
## 时间序列预测模型
# 以提取第一个组合为例，然后进行预测
a = table1.loc[(table1['seller_no1']==index.iloc[0,0]) &
                (table1['product_no1']==index.iloc[0,1]) &
                (table1['warehouse_no1']==index.iloc[0,2])]
qty = a['qty'].values
qty
### 平稳性检验
'''
1，平稳性检验
图解法：时序图&自相关图
假设检验：ADF 单位根检验&KPSS 单位根检验

'''
plt.plot(qty, 'r-*')
plt.title('原始数据的时序图')
plt.show()

plt.plot(np.diff(qty), 'r-*')
```

```
plt.title('一次差分后数据的时序图')
plt.show()
adf = adfuller(qty)
print(adf)
# (-1.725445177524298, 0.41805232968970923, 9, 12, {'1%': -4.137829282407408, '5%':
-3.1549724074074077, '10%': -2.7144769444444443}, 46.92397036481875)
# p=0.41>0.05 接受原假设，存在单位根，非平稳
adf = adfuller(np.diff(qty, 1))
print(adf)
### 白噪声检验
"""
2，白噪声检验

"""
# 看延迟 6 期 12 期的 p 值小于 0.05，拒绝原假设，非白噪声
lb = lb_test(qty, lags=12)
print(lb)
### 模型的定阶
"""
3，模型的定阶&参数的估计

"""
fig = plt.figure(figsize=(12, 6))
ax1 = plt.subplot(121)
plot_acf(np.diff(qty, 1), ax=ax1, title='自相关')
ax2 = plt.subplot(122)
# 有报错，所以加了 lags=8
plot_pacf(np.diff(qty, 1), ax=ax2, title='偏自相关')
plt.show()
for i in range(0, 3):
    for j in range(0, 3):
        md = sm.tsa.ARIMA(qty, order=(i, 1, j)).fit()
        print([i, j, md.aic, md.bic])
# 从 AIC，BIC 角度选择 p=0,q=2（但参数估计不行）
# 从 ACF,PACF 角度选择 p=2,q=1
# 从参数(2,3,0)更合适
zmd = sm.tsa.ARIMA(qty, order=(1, 1, 2)).fit()
# 显示模型的所有信息
print(zmd.summary())
### 模型的显著性检验
"""
4，模型的显著性检验/残差检验

"""
fig, ax = plt.subplots(1, 2)
# 设置画布大小为宽度：12 英寸，高度：6 英寸
```

```
fig.set_size_inches(12, 6)
residuals = pd.DataFrame(zmd.resid)
residuals.plot(title='残差', ax=ax[0])
residuals.plot(kind='kde', title='密度', ax=ax[1])
plt.show()
# 看延迟 6 期 12 期的 p 值小于 0.05，拒绝原假设，非白噪声
lb = lb_test(residuals, lags=12)
print(lb)
# 残差是白噪声
### 模型预测
"""
5，讨论 ARIMA 时间序列的预报

"""
fig= plt.figure(figsize=(12, 6))
plt.plot(qty[1:], 'o-k')
plt.plot(qty[1:]+zmd.resid, 'p--')
plt.legend(['原始值', '预测值'])
# 预测未来 15 天数据
dnext = zmd.forecast(15)[0]
print(dnext)
plt.show()
# 计算评价指标
print(1-sum(abs(zmd.resid))/sum(qty))
### 计算 1996 种组合预测值
date = [f'2023-05-{i}' for i in range(16, 31)]
Date, Seller, Product, Ware, Dnext = [], [], [], [], []
num=0
for i in range(index.shape[0]):
    num+=1
    print(f'第{num}个组合正在预测...')
    try:
        a = table1.loc[(table1['seller_no1']==index.iloc[i,0]) &
                       (table1['product_no1']==index.iloc[i,1]) &
                       (table1['warehouse_no1']==index.iloc[i,2])]
        qty = a['qty'].values
        zmd = sm.tsa.ARIMA(qty, order=(1, 1, 2)).fit()
        dnext = zmd.forecast(15)[0]
        Dnext.extend(dnext)
        Date.extend(date)
        Seller.extend(['seller_' + str(index.iloc[i,0])] * 15)
        Product.extend(['product_' + str(index.iloc[i,1])] * 15)
        Ware.extend(['warehouse_' + str(index.iloc[i,2])] * 15)
    except:
        continue
```